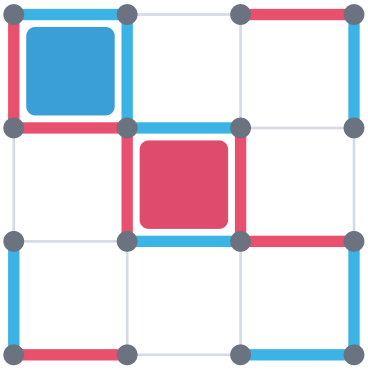
SEMESTER AI PROJECT · TEAM OF FIVE · 20 DAYS

Dots and Boxes AI

A web game where you play Dots and Boxes against AI bots of different strengths. The game adjusts to your level, gives hints, and explains your mistakes.

Minimax + alpha-beta | Monte Carlo simulation | Adaptive difficulty | LLM explanations | Bot experiments



01 What we are building

A playable web game with AI bots that think in different ways, a difficulty system that adapts to the player, and a hint and review feature. Our report compares how well each AI approach plays.

PLAY
Play against a bot on Easy, Medium or Hard, or let the game pick a level for you.

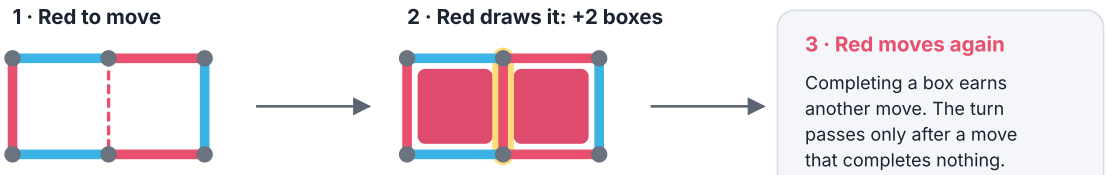
LEARN
Ask for a hint, see risky moves highlighted, and read a short review after the game.

COMPARE
Bots play each other many times, and we compare their win rates.

REPORT
What worked, what didn't, and why, backed by the numbers.

02 The game in 30 seconds

- Players take turns drawing a line between two neighbouring dots.
- Whoever draws the **fourth side of a box** claims it and **takes another turn**.
- When all lines are drawn, the player with more boxes wins.



Both boxes have 3 sides. The middle edge (dashed) finishes both.

One edge can complete two boxes. Red scores both.

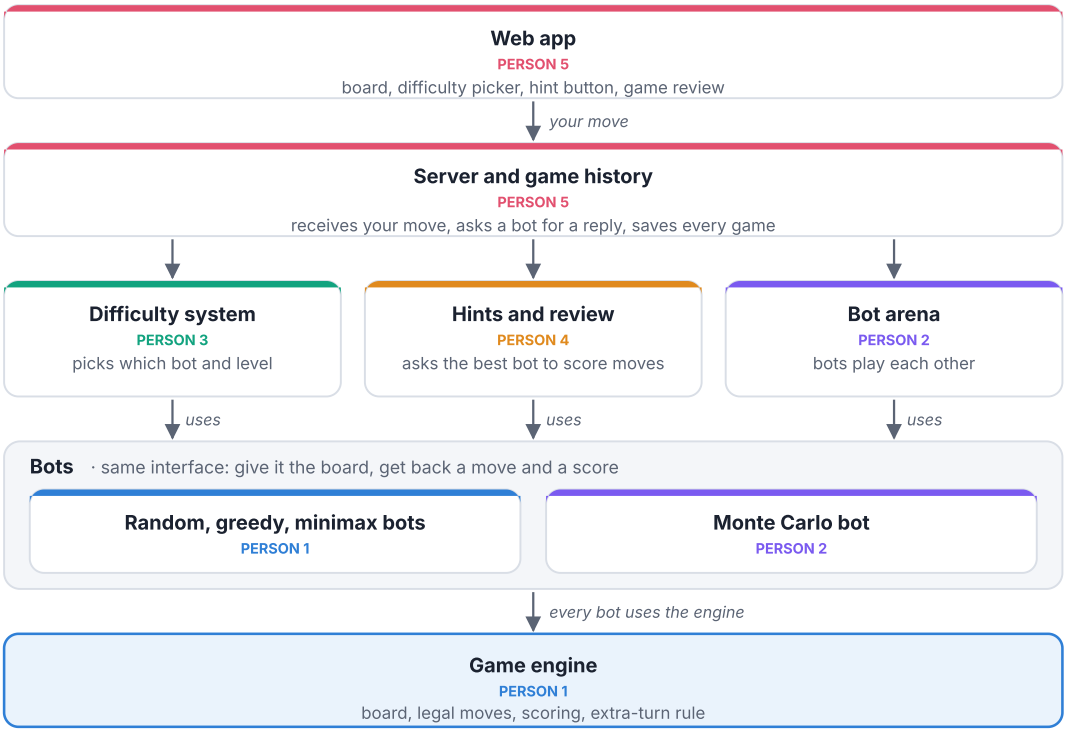
The extra-turn rule. Completing a box means you move again, so turns do not simply alternate. Every bot has to handle this.

WHY IT IS A GOOD AI PROBLEM

The rules are simple, but grabbing every box you can is often a trap: it can force you to give the opponent a long row of boxes later. A good AI has to look ahead, not just play greedily. A full board also has far too many positions to check them all, so the bots need smart search or learning.

03 How the parts fit together

Everyone builds on one shared **game engine**. Every bot works the same way: give it the board, and it returns a move plus a score saying how good that move is. Any bot can then be plugged into the game, the difficulty system or the hint feature without extra work.



Architecture. The web app talks to a small server. The server asks a bot for its move, and every bot uses the same game engine. Colours show who owns each part.

04 Team split

Each person owns one part, with a clear goal for the 20 days. Stretch goals are optional, only if there is time left.

PERSON 1

Game engine and classic AI bot

1

Builds the core everyone else depends on: the game rules in code, plus the first "thinking" bot, which looks a few moves ahead.

WHAT TO BUILD

- **Game engine:** board, legal moves, scoring and the extra-turn rule, with tests to catch rule bugs early.
- **Random and greedy bots:** a random bot, and a bot that takes a box whenever it can and otherwise avoids giving one away. These are our baselines.
- **Minimax bot with alpha-beta pruning:** looks a few moves ahead and skips branches that cannot matter. It scores positions with a simple rule: boxes won minus boxes lost.
- **Search depth setting:** deeper search plays stronger, which gives Person 3 an easy way to make difficulty levels.

Python · Minimax · Alpha-beta pruning · Unit tests

Works with: everyone, since all bots and features use the engine. Share it by day 4 so others can start.

Done when the minimax bot beats the greedy bot in most games. **Stretch:** a smarter scoring rule.

PERSON 2

Monte Carlo bot and bot arena

2

Builds a second kind of AI that learns by trial instead of looking ahead, then runs the bot-vs-bot experiments that show which approach plays best.

WHAT TO BUILD

- **Monte Carlo bot:** for each possible move, play many quick random games to the end and pick the move that wins most often. No look-ahead tree is needed.
- **Smarter playouts:** make the simulated games follow the greedy rule instead of being fully random, so the bot plays better with the same effort.
- **Strength setting:** more simulations means a stronger bot, which gives Person 3 a second easy way to make difficulty levels.
- **Bot arena:** a script where every bot plays every other bot about 100 times, producing a win-rate table.
- **Results charts:** win rate and thinking time for each bot and setting, for the report.

Monte Carlo simulation · Python · matplotlib

Works with: Person 1 (engine and minimax bot), Person 3 (simulations as a difficulty setting), Person 5 (results for the report and demo).

Done when the Monte Carlo bot beats the greedy bot, and the arena table compares all bots. **Stretch:** upgrade to full MCTS (UCT), which reuses earlier simulations.

PERSON 3

Adaptive difficulty

3

Makes the game fun for every player: matches them with a bot of the right strength and adjusts as they get better or worse.

WHAT TO BUILD

- **Difficulty levels:** Easy, Medium and Hard (up to 5), each a setting of an existing bot, for example greedy with some random moves, minimax at depth 2, or Monte Carlo with many simulations.
- **Player rating:** a simple Elo score for each player that goes up when they win and down when they lose.
- **Auto-adjust:** after each game, move the player up or down a level so they win roughly half the time.
- **Player stats:** win rate, games played and how often they give away boxes, shown on a profile screen.

Elo rating · Difficulty adjustment · Player stats

Works with: Persons 1 and 2 (bots to build levels from, plus arena results to rank the levels), Person 5 (game history and the profile screen).

Done when a test player who always loses gets moved down, and one who always wins gets moved up. **Stretch:** mistakes that look more human.

PERSON 4

Hints and game review

4

Helps players learn by showing what the AI is thinking: suggesting moves, warning about bad ones, and explaining mistakes in plain English.

WHAT TO BUILD

- **Hint button:** asks the strongest bot for its best move and highlights it on the board.
- **Danger highlight:** marks lines that would leave a box with 3 sides, which gives the opponent a free box.
- **Move review:** after the game, labels each move Good, Okay or Mistake by comparing it with the bot's best move.
- **LLM explanations:** sends the 2–3 biggest mistakes, with the facts from the review, to an LLM that explains them in a sentence or two.

Explainable AI · Move evaluation · LLM API · Prompting

Works with: Persons 1 and 2 (bots that score moves), Person 5 (showing hints and reviews in the app).

Done when hints and the review work in the app, and the explanations match what actually happened. **Stretch:** a heatmap of good and bad moves.

PERSON 5

Web app, integration and testing

5

Builds what people actually see and use, and connects everyone's work into one app.

WHAT TO BUILD

- **Web app:** a clickable board like the screenshot, with scores, turn indicator, difficulty picker, hint button and review screen.
- **Small server:** a FastAPI backend that receives the player's move and returns the bot's move.
- **Game history:** saves every game (moves, result, level) to a file or SQLite, for Persons 3 and 4.
- **Shared repo:** GitHub setup and a weekly "does everything still run together" check.
- **Demo setup:** one command to start the whole app, ready for the final presentation.

HTML/JS or React · FastAPI · SQLite · GitHub

Works with: everyone. Plugs in every bot and feature, and shares game data with Persons 3 and 4.

Done when anyone can open the app, pick a level and play a full game. **Stretch:** a quick feedback survey with classmates.

05 20-day plan

Four short phases. Each ends with a checkpoint the whole team checks together.

1 · Set up days 1–3	2 · Build days 4–11	3 · Connect days 12–16	4 · Finish days 17–20
Repo created, engine design agreed, everyone knows their task	Each person builds their part alone, using the engine and the greedy bot	All parts plugged into the app; bug fixing	Arena results, report, demo practice
CHECKPOINT Two random bots can play a full game	CHECKPOINT Each part works on its own	CHECKPOINT A person can play, get hints and see a review	CHECKPOINT Report written and demo rehearsed